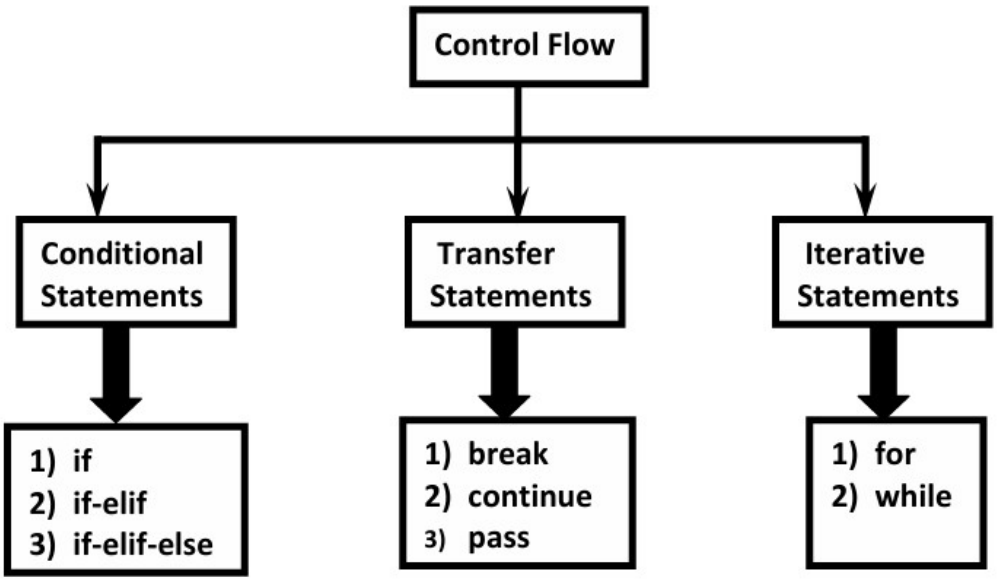
Flow Control

***Control flow statements** are fundamental components of programming languages that allow developers to control the order in which instructions are executed in a program. They enable execution of a block of code multiple times, execute a block of code based on conditions, terminate or skip the execution of certain lines of code, etc.*

Control flow refers to the order in which statements within a program execute. While programs typically follow a **sequential flow** from top to bottom, there are scenarios where we need more flexibility.



1) if

```
if condition :
    statement

    OR

if condition :
    statement-1
    statement-2
    statement-3
```

If condition is true then statements will be executed.

Example:

```
# Prompt the user to enter a number
# The input is converted to an integer (whole number)

number = int(input("Enter a whole number: "))
print(f"You entered: {number}")
# Check the properties of the number using only if statements
if number > 0:
   print("The number is positive.")
if number < 0:
   print("The number is negative.")
if number == 0:
   print("The number is zero.")
if number % 2 == 0:
   print("The number is even.")
if number % 2 != 0:
   print("The number is odd.")
print("Checks complete.")
```

2) if-else

```
if condition:
        Action-1
else:
        Action-2
```

Example

```
# Program to check if a student passes or fails based on their score

# Get the student's score as input
score_input = input("Enter the student's score (0-100): ")

# Convert the input string to an integer
score = int(score_input)

# Check if the student passes or fails using if-else conditions
if score >= 60:
   print("The student passes!")
else:
   print("The student fails.")
```

3) if-elif-else

```
if condition1:
        Action-1
elif condition2:
        Action-2
elif condition3:
        Action-3
elif condition4:
        Action-4
else:
       Default Action
```

Based condition the corresponding action will be executed.

Example:

```
# Get input from the user
# int(input(...)) converts the user's text input into an integer (whole number)
number = int(input("Enter a number: "))

# Check the conditions sequentially
if number > 0:
   print(f"The number {number} is positive.")
elif number < 0:
   print(f"The number {number} is negative.")
else:
   print(f"The number {number} is zero.")

print("This statement is always executed, as it is outside the if-elif-else block.")
```

Ques:

Write a Program to find Biggest of given 3 Numbers from the Command Prompt?

```
n1=int(input("Enter First Number: "))
n2=int(input("Enter Second Number: "))
n3=int(input("Enter Third Number: "))
if n1>n2 and n1>n3:
   print(f"Biggest Number is {n1}”)
elif n2>n3:
   print(f"Biggest Number is {n2}”)
else :
   print("Biggest Number is:",n3)
```

Ques:

Write a Program to Check whether the given Number is in between 10 and 100?

```
n=int(input("Enter Number: "))
if n>=10 and n<=100 :
   print(f"The number {n} is in between 1 to 100")
else:
   print(f"The number {n} is not in between 1 to 100")
```

Ques

Determining if a Year is a Leap Year?

Note :
A year is a leap year if it is divisible by 4, except for years divisible by 100 but not by 400.

```
year = int(input("Enter a year: "))

if (year % 400 == 0) and (year % 100 == 0):
    print(f"{year} is a leap year.")
elif (year % 4 ==0) and (year % 100 != 0):
    print(f"{year} is a leap year.")
else:
    print(f"{year} is not a leap year.")
```

Try giving input as -> 1900, 2017, 2056, 2026, 1365

Ques:

Suppose we need to assign different grades to students based on their scores.

1. If a student scores above **90**, assign grade **A**
2. If a student scores above **75**, assign grade **B**
3. If a student scores above **65**, assign grade **C**

```
score = float(input("Enter the student's score: "))
if score > 90:
    grade = 'A'
elif score > 75:
    grade = 'B'
elif score > 65:
    grade = 'C'
else:
    grade = 'Below C'
print(f"The assigned grade is: {grade}")
```

Iterative Statements

If we want to execute a group of statements multiple times then we should go for Iterative statements.

Python supports 2 types of iterative statements.

1) for loop
2) while loop

1) for loop:

If we want to execute some action for every element present in some sequence (it may be string or collection) then we should go for for loop.

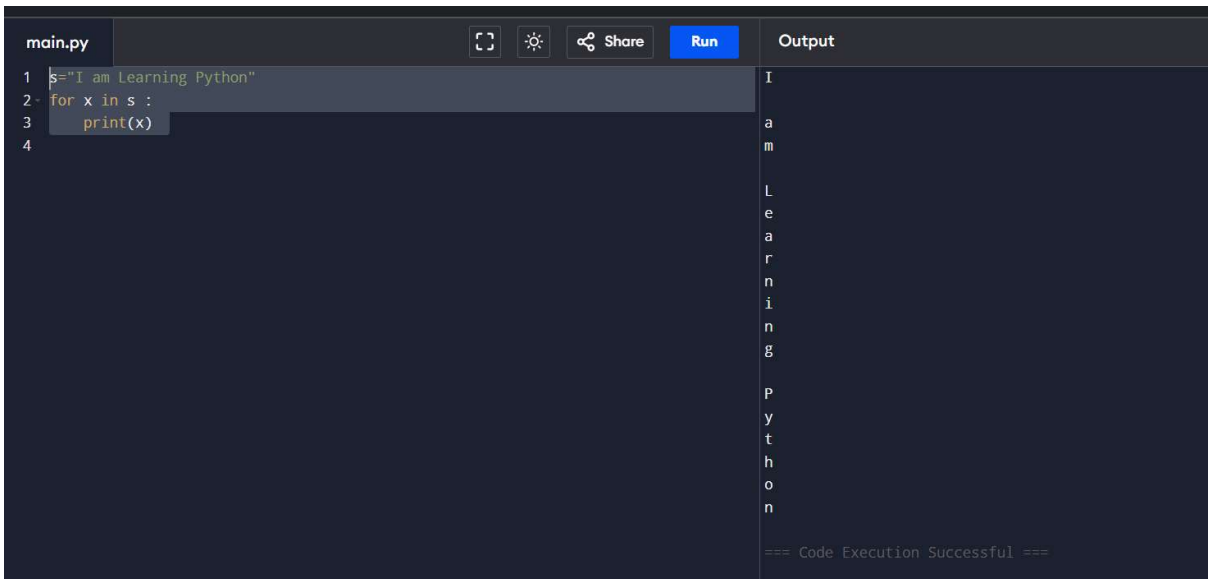
Syntax:

```
for x in sequence:
        Body
```

Example:

To print characters present in the given string?

```
s="I am Learning Python"
for x in s :
    print(x)
```



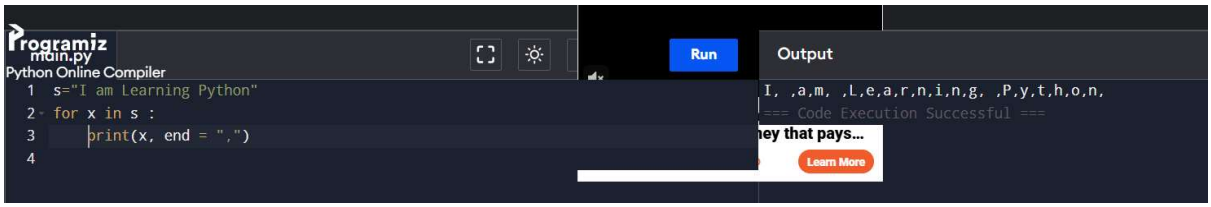
Here We can see output, characters are printed line by line.

If we want in same line then,

Include **end = “”** inside print statement.

Note:-> you can give empty space, space, comma, colon, etc.

print(x, end = ",")



Example

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
  print(fruit)
```

Example

```
# Prints numbers from 0 to 4.

for i in range(5):
  print(i)
```

Example

```
# iterate from i = 0 to i = 3

for i in range(0, 4):

  print(i)
```

Ques:

Write a program to display odd numbers from 0 to 20?

```
for x in range(21) :
    if (x%2!=0):
        print(x)
```

Ques:

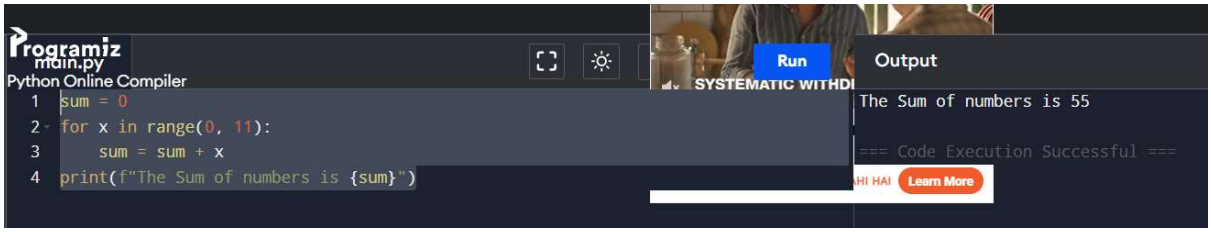
Write a program to display numbers from 10 to 1 in descending order?

```
for x in range(10,0,-1):
        print(x)
```

Ques:

To print sum of numbers present inside list?

```
sum = 0
for x in range(0, 11):
    sum = sum + x
print(f" The sum of numbers is {sum}")
```



2) while loop:

If we want to execute a group of statements iteratively until some condition false, then we should go for while loop.

Syntax:

while condition:

    body

Example:

To print numbers from 1 to 10 by using while loop?

```
x = 1
while x <= 10:
    print(x)
    x = x+1
```

Example:

To display the sum of first n numbers

```
n=int(input("Enter number:"))
sum=0
i=1
while i<=n:
    sum=sum+i
    i=i+1
print(f"The sum of first {n} numbers is :-> {sum}")
```

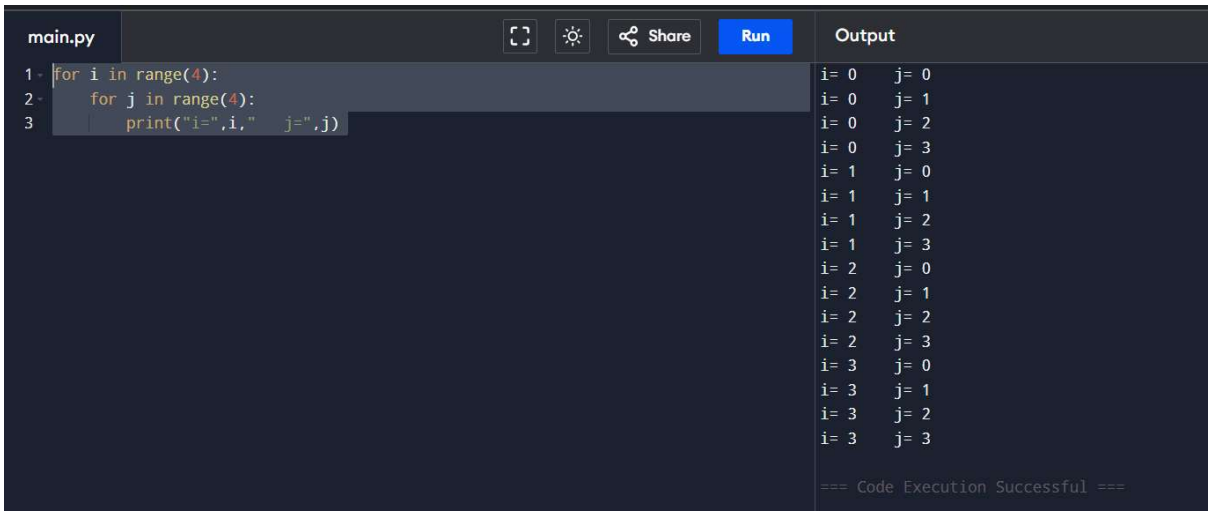
Infinite Loops:

```
i=0;
while True :
    i=i+1;
print("Hello",i)
```

Nested Loops:

Sometimes we can take a loop inside another loop, which are also known as nested loops.

```
for i in range(4):
    for j in range(4):
        print("i=",i,"   j=",j)
```



Transfer Statements

(1) **Break**

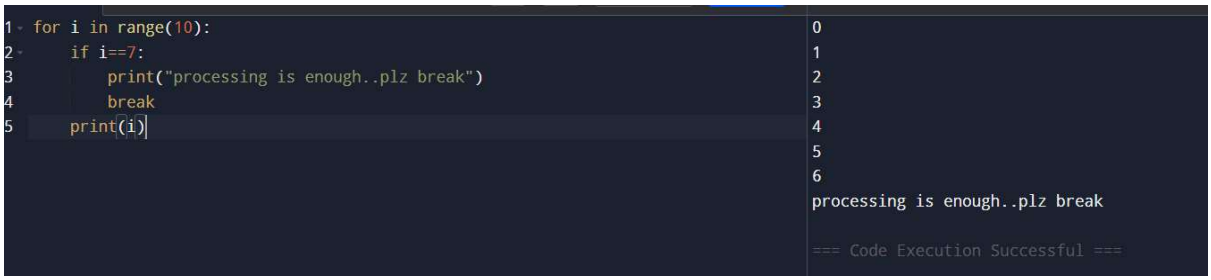
(2) **Continue**

1) break:

We can use break statement inside loops to break loop execution based on some condition.

Example:

```
for i in range(10):
    if i==7:
        print("processing is enough..plz break")
        break
    print(i)
```



Example:

```
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    if num == 3:
        print("Break condition met! Exiting the loop.")
        break  # Exits the loop entirely
    print(f"Checking number: {num}")
print("Loop ended.")
```

2) continue:

We can use continue statement to skip current iteration and continue next iteration.

Example:

To print odd numbers in the range 0 to 9

```
for i in range(10):
    if i%2==0:
        continue
    print(i)
```

Example:

```
cart=[10,20,500,700,50,60]
for item in cart:
    if item>=500:
        print("We cannot process this item :",item)
        continue
    print(item)
```

Example:

```
numbers=[10,20,0,5,0,30]
for n in numbers:
    if n==0:
        print("Hey how we can divide with zero..just skipping")
        continue
    print(f"{100/n}")
```

Loops with else Block:

- Inside loop execution, if break statement not executed, then only else part will be executed.
- else means loop without break.

Example:

```
cart=[10,20,30,40,50]
for item in cart:
   if item>=500:
      print("We cannot process this order")
      break
   print(item)
else:
   print("Congrats ...all items processed successfully")
```

Example:

```
cart=[10,20,600,30,40,50]
for item in cart:
   if item>=500:
      print("We cannot process this order")
      break
   print(item)
else:
   print("Congrats ...all items processed successfully")
```

main.py

```python
cart=[10,20,600,30,40,50]
for item in cart:
    if item>=500:
        print("We cannot process this order")
        break
    print(item)
else:
    print("Congrats ...all items processed successfully")
```

Output

```
10
20
We cannot process this order

=== Code Execution Successful ===
```

pass statement:

pass is a keyword in Python.

- In our programming syntactically if block is required which won't do anything then we can define that empty block with pass keyword.

Example:

```python
for i in range(100):
    if i%9==0:
        print(i)
    else:
        pass
```

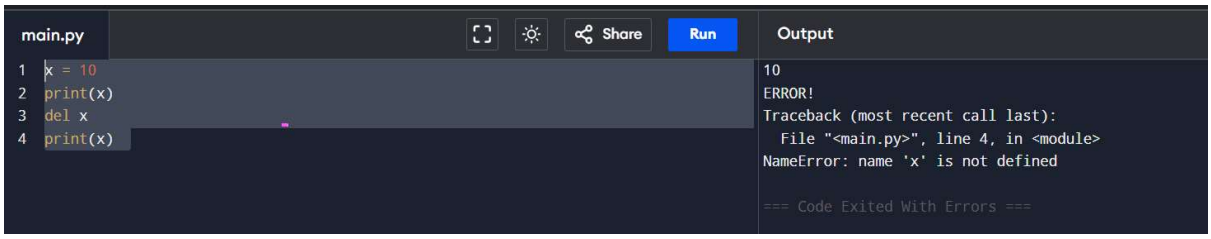
del Statement:

del is a keyword in Python.

- **After using a variable, it is highly recommended to delete that variable if it is no longer required, so that the corresponding object is eligible for Garbage Collection.**
- **We can delete variable by using del keyword.**

Example:

```
x = 10
print(x)
del x
print(x)
```



Example:

```
x = 10
print(x)
x = None
print(x)
```

Ques:

Write a program to find factorial.

The factorial of a number is the product of all the integers from 1 to that number.

For example, the factorial of 6 is 1*2*3*4*5*6 = 720

```
num = int(input("Please provide integer value:-> "))
factorial = 1
# check if the number is negative, positive or zero
if num < 0:
  print("Sorry, factorial does not exist for negative numbers")
elif num == 0:
  print("The factorial of 0 is 1")
else:
  for i in range(1, num + 1):
     factorial = factorial*i
  print(f"The factorial of {num} is {factorial}”)
```

Ques:

write a program to find fibonacci series in python

 # Program to display the Fibonacci sequence up to n-th term

```
nterms = int(input("How many terms? "))
# first two terms
n1, n2 = 0, 1
count = 0
# check if the number of terms is valid
if nterms <= 0:
  print("Please enter a positive integer")
# if there is only one term, return n1
elif nterms == 1:
  print("Fibonacci sequence upto",nterms,":")
  print(n1)
# generate fibonacci sequence
else:
  print("Fibonacci sequence:")
  while count < nterms:
     print(n1)
     nth = n1 + n2
     # update values
     n1 = n2
     n2 = nth
     count += 1
```

main.py

Share Run

```python

nterms = int(input("How many terms? "))

# first two terms
n1, n2 = 0, 1
count = 0

# check if the number of terms is valid
if nterms <= 0:
   print("Please enter a positive integer")
# if there is only one term, return n1
clif nterms == 1:
   print("Fibonacci sequence upto",nterms,":")
   print(n1)
# generate fibonacci sequence
else:
   print("Fibonacci sequence:")
   while count < nterms:
       print(n1)
       nth = n1 + n2
       # update values
       n1 = n2
       n2 = nth
       count += 1

```

Output

```
How many terms? 5
Fibonacci sequence:
0
1
1
2
3

=== Code Execution Successful ===
```